

基于 OpenHands SDK 的 Multi-Agent FPGA 自动化设计系统 — 汇报稿

汇报人：贾阔源 指导老师：刘亚男 日期： 年 月

一、演示文稿概述

本次毕业设计汇报共包含 页演示文稿，按照 研究背景及意义 关键技术框架 项目具体实施 总结与展望 四个章节依次展开。全文配有 张图片，其中包括 张核心技术图（架构图、流程图、状态机图、数学 导图等），其余为每页右上角重复出现的校徽标识。整个演示的核心论点围绕三大创新展开：框架控制的编排状态机、四层分级验证体系以及 六层 约束引擎。以下按章节顺序对每页内容及其图片进行详细解读。

二、第一章：研究背景及意义（Slides 1-5）

2.1 封面与目录

演示文稿以一张简洁的封面页开篇，标题 基于 的 自动化设计系统 居中排列，下方标注汇报人与指导老师信息，上方放置校徽标识。整体采用四角装饰矩形搭配圆角主框架的学术风格排版。

紧随其后的目录页将汇报内容划分为四个部分，以编号圆形图标配合文字说明的方式呈现，布局清晰直观，为听众建立了完整的汇报路线图。

2.2 大语言模型发展时间线（Slide 3）

第三页通过一条纵向时间线，梳理了大语言模型从 世纪至今的关键发展节点。时间线以垂直中轴线为主干，左右交替排列八个里程碑事件：从早期基于统计的 模型出发，经过 年 团队提出的神经网络语言模型、 年 框架和 年 机制，直到 年 提出 架构 这被标注为 革命性突破，也是后续所有大模型的基石。此后 年 和 相继问世， 年国产大模型 横空出世， 年则被标注为 智能体工程元年。

这条时间线的叙事意图非常明确：通过回溯技术演进史，将本课题锚定在 年 框架爆发的时代背景之中，说明课题立项既有深厚的技术积淀，又恰逢其时。

2.3 Harnessing Engineering — 驾驭工程（Slide 4）

第四页引入了本课题的一个核心概念（**自动驾驶工程**），并以一张极具说服力的对比图加以论证。这张图采用左右分栏布局，左侧展示了**传统工作流**，右侧展示了**优化工作流**。

在左侧流程中，**生成代码**后需要经历三轮人工审查：第一轮发现类型错误，**修复后**第二轮又发现格式问题和引用错误，再次修复后第三轮才最终通过。整个过程消耗约**30分钟**，且始终依赖人工介入，标注为**低效率**。左侧各节点使用红色暖色调，视觉上暗示了人工审查带来的效率瓶颈。

右侧流程则截然不同：**生成代码**后，**自动化机制**自动执行**静态分析**和**动态测试**，以结构化格式一次性反馈所有错误，**据此**一轮修复完毕，再由**自动化机制**确认所有检查通过。整个过程仅消耗约**5分钟**，完全无需人工介入。右侧节点使用绿色冷色调，传达了自动化的高效感。图片底部的总结一语中的：
“**自动化机制**，是自动驾驶工程的核心。”

这张对比图的深层含义在于揭示了本课题面临的根本挑战：大语言模型本质上是不可靠的，它会产生幻觉（生成不存在的端口名）、发生**任务漂移**（偏离指定任务范围）、还可能在验证未完成时提前宣布成功。**自动驾驶工程**作为本课题提出的解决方案，通过六层结构化约束（**需求层、设计层、实现层、验证层、部署层、维护层**），从外部工程化地约束**大语言模型**的行为，而非依赖模型的**自律性**。这也是本课题的第三个核心创新点。

2.4 LLM 在 EDA 领域的应用（Slide 5）

第五页聚焦文献综述，通过两张论文引用图和一段文字说明，完成了对当前研究前沿的定位。

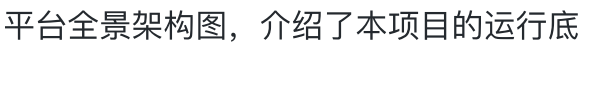
第一张图来自**IEEE Transactions on Computer-Aided Design and Integration of Circuits**论文，展示了**自动驾驶工程**领域的系统架构。图中采用三列布局：左侧是**需求层**，包含多个**需求项**和**约束项**；中间是**设计层**，承载**设计规则**、**设计模板**和**设计工具**三个功能模块；右侧是具体的**工具链**，涵盖**仿真器**、**综合器**、**布局器**、**物理验证器**等从仿真到物理实现的完整工具。各层之间通过**数据流**和**控制流**通信，**设计层**通过发送上下文和工具请求与**工具链**交互，实现了从**需求**到**物理实现**的自动化流程。

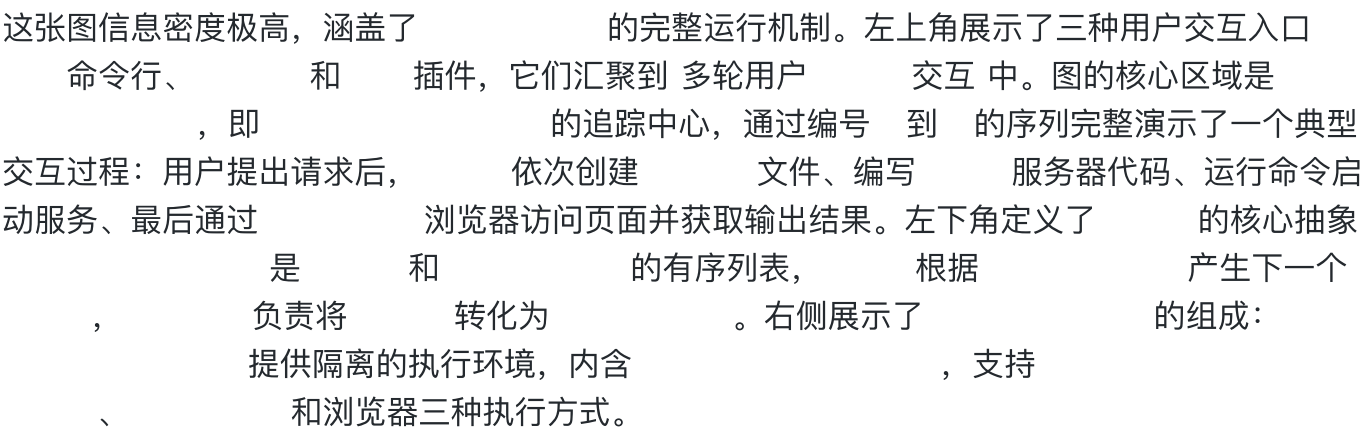
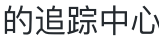
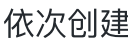
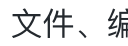
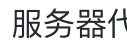
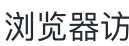
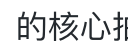
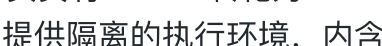
第二张图来自**IEEE Transactions on Computer-Aided Design and Integration of Circuits**论文，展示了端到端的**自动驾驶工程**自动生成流程。图中用三种颜色的色块划分了三个阶段：红色的**需求理解**（理解设计规格并提取架构）、绿色的**代码生成**（生成代码并通过工具实现）、蓝色的**自我审查**（对生成结果进行自我审查和修正）。图上方还特别标注了竞品对比：**竞品A**和**竞品B**仅自动化了**需求理解**阶段，**竞品C**和**竞品D**仅自动化了**代码生成**阶段，而**自动驾驶工程**实现了三个阶段的端到端全覆盖，并通过**自我审查**和**需求理解**两条反馈路径形成迭代闭环。

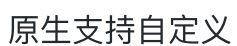
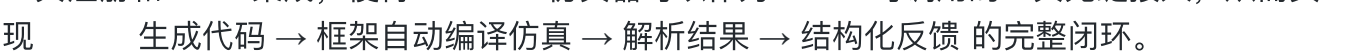
页面下方列出了文字说明，将**自动驾驶工程**在**EDA**领域的应用归纳为**需求理解**、**代码生成**、**自我审查**三个维度，同时指出现有方案普遍存在模块间依赖处理不足、验证层级单一和**约束不够**等问题，这恰构成了本课题的创新切入点。

三、第二章：关键技术框架（Slides 6-11）

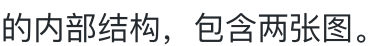
3.1 OpenHands 平台概览（Slide 7）

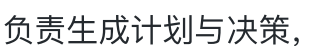
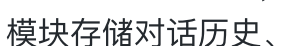
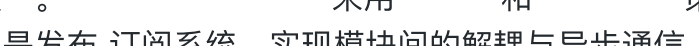
进入第二章后，第七页以一张占满全页的  平台全景架构图，介绍了本项目的运行底座。

这张图信息密度极高，涵盖了  的完整运行机制。左上角展示了三种用户交互入口：命令行、 和  插件，它们汇聚到  多轮用户交互中。图的核心区域是 ，即  的追踪中心，通过编号  到  的序列完整演示了一个典型交互过程：用户提出请求后， 依次创建  文件、编写  服务器代码、运行命令启动服务、最后通过  浏览器访问页面并获取输出结果。左下角定义了  的核心抽象，是  和  的有序列表， 根据  产生下一个 ，负责将  转化为 。右侧展示了  的组成： 提供隔离的执行环境，内含 ，支持 、 和浏览器三种执行方式。

选择  而非  等其他框架的核心原因在于： 原生支持自定义工具注册和  集成，使得  仿真器可以作为  可调用的工具无缝接入，从而实现  生成代码 → 框架自动编译仿真 → 解析结果 → 结构化反馈 的完整闭环。

3.2 OpenHands SDK 模块细节（Slide 8）

第八页进一步展开了  的内部结构，包含两张图。

第一张是  九大核心模块的功能对照表。 负责生成计划与决策，通过  和自然语言反馈驱动，支持  和  模式。 模块存储对话历史、执行结果和外部知识，支持  检索和记忆压缩，解决了上下文窗口有限的难题。 提供可配置的规则  动态混合规划器。 通过  实现  输出到结构化工具调用的映 。 采用  和  策略对模型行为进行检查与修正。 是发布  订阅系统，实现模块间的解耦与异步通信。 通过  创建隔离的工作空间，保证执行的安全性和可复现性。 是框架的  指挥官，负责迭代循环、状态管理和子  调度。 提供  接口，管理会话的创建、恢复和持久化。

第二张是后端架构图，清晰展示了从用户到运行时的完整数据通路：用户通过前端单页应用发起请求，经  到达  后端，后端分别连接事件流、存储和大语言模型提供者。 区域以绿色背景标识，内部通过运行时接口分发到  运行时、本地运行时或远程运行时，最终到达动作执行服务器，支持  会话、 插件和浏览器环境三种执行方式。

基于这些核心模块，本项目构建了五种角色：驾驭全局流程、负责终态总结、执行单模块的生成、处理结构化修复、执行鲁棒性测试。这种分层委派设计使编排器专注于全局决策，工作器专注于局部执行。

3.3 DeepSeek 架构深度解析 (Slides 9-11)

第九页是全部演示中信息密度最高的一页，包含多达 张技术图，完整剖析了 大语言模型的架构特点。

首先是 的整体架构图。左侧展示了标准的 堆叠结构：输入经过 进入 模块，再经 进入 ，两处均有残差连接。右上角展示了 的核心设计：输入的 经过 模块，由 机制动态选择要激活的专家网络。专家分为绿色标注的 （所有 共享）和蓝色标注的 （按需激活），各专家输出经加权求和后产生最终结果。右下角展示了 （ ）机制的细节：输入被投影到低维潜在空间（分别得到 和 ），经过展开、 位置编码和拼接后参与多头注意力计算，其中 的潜在表示在 理时被缓存。

接下来是 模型与 模型的对比图。 模型的每个 使用单一的 层，所有参数在每次 理中全部激活。而 模型则将 替换为 加多专家结构： 根据输入动态路由，仅激活其中少量专家（蓝色高亮标识）， 则始终参与计算。这种稀疏计算的本质可以用页面上另一张直观的概念图来概括： 一个稠密大矩阵变成数百个稠密小矩阵，每次只激活 到 个。

参数分布表进一步量化了这一架构的特点： 总共拥有 参数，其中 模块占 （ ）， 单独就占了 。相比之下， 模块仅占 （ ）， 和 层加起来不到 。这意味着模型的绝大部分容量存储在数百个专家网络中，但每次 理只激活其中极小一部分，从而在保持模型总能力的同时大幅降低计算成本。

的数学流程图和公式图则详细展示了 压缩的数学原理。输入 分两路处理：一路通过 和 的转置相乘得到 的注意力分数，另一路通过 投影到低维空间后缓存。注意力计算完成后，通过 和 输出最终结果。公式中明确标注了关键变量： 是压缩后的 潜在表示， 和 分别是 从压缩空间恢复的 和经过 编码的位置 ，两者拼接后参与注意力计算。这种设计的核心优势是：只需缓存低维的潜在表示而非完整的 矩阵，大幅降低了长上下文 理的显存占用。

第十和第十一页分别深入讲解了 理的两个阶段。 阶段处理完整的输入序列，所有 并行计算，张量维度从输入的 经 压缩到 ，再展开为 个注意力头。 阶段则逐 自回归生成，序列维度从 缩减为 ，同时引入 机制，将当前步的 与历史缓存拼接。这两个阶段的分离对 系统有直接的工

程意义：编排器的系统提示（包含长篇的合约文档和技能描述）主要消耗 算力，而在修复循环中的交互式 理主要消耗 算力。 的 压缩机制使得这种长上下文、多轮对话的场景在算力和显存上都更加高效。

四、第三章：项目具体实施（Slides 12-19）

4.1 系统六层架构（Slide 13）

第十三页以一张全幅横向的架构图展示了系统的整体分层设计。图中采用自顶向下的布局，以浅黄色背景区分每一层的边界，淡紫色方框标注具体的文件和模块。

最顶层是（ ），提供 、 、 和 五个命令入口。 下方是 ，包含（ 导入垫片）、（运行时初始化）、（ 工厂）、（对话驱动器）、（执行会话）、（上下文组装）和（自定义工具注册）七个模块。 向下分叉为三个平行的子层： 包含角色规格定义、提示词构建、提示词合约、技能引用和记忆存储五个组件； 包含执行器、适配器和 集成三个组件； 包含综合引擎、数据契约、策略定义、执行器合约和工作区生成五个组件。

这种分层设计的核心优势在于关注点分离：每一层只负责一个维度的职责，层间通过 模型定义的严格数据契约通信，所有契约均使用 参数拒绝未定义字段，从数据层面保证了系统的正确性。

4.2 端到端工作流程（Slide 14）

第十四页通过两张互补的图详细展示了系统的运行流程。

第一张是六方参与者的时序图，完整呈现了一次批次执行的端到端交互过程。首先选择当前要执行的 ，然后创建 并注入消息。 调用 启动门控轮询。在对话过程中， 调用 自定义工具并传入任务 路径。 将请求派发到对应的 执行流程，后者通过 依次执行 和 操作。仿真结果以 的形式逐层回传，最终构建为结构化的 反馈给 。 在收到结果后，比对 并执行 ，决定是否进入下一个 。

第二张是完整的编排状态机 图，展示了系统的十个状态和它们之间的转移关系。流程从 开始，依次经过 和 （均使用 理配置且禁止委派子 ），然后进入 、 和 （切换为 执行配置并允许委派模块工作者）。如果模块需要 鲁棒性测试，则进入

状态；否则直接进入，随后执行
(回到配置)，最终到达终态。

这两张图共同论证了本课题的第一个核心创新点：框架控制与提示词控制的根本区别。在等传统方案中，执行进度由的输出决定模型说验证通过了系统就继续进，这属于架构。而在本系统中，状态机拥有控制权，每一步状态转移必须满足框架定义的前置条件（如所有通过），无法绕过验证门控提前宣布成功。

4.3 任务规划与知识注入 (Slide 15)

第十五页展示了两个关键机制：可组合的提示词架构和拓扑规划。

第一张图以色彩编码的方式展示了提示词的组合架构。最上方红色边框的定义了五条不可违反的硬约束：界定设计范围、约束输入输出格式、禁止直接调用、确立技能文件的权威性、声明框架对进度的控制权。第二行蓝色边框的定义了九条可按阶段组合的软约束，覆盖了从生成到验证到修复到测试的各个执行阶段。中间黄色边框的提供了每次任务的实例化信息，如可写文件列表和模块特有的。下方绿色边框的和分别注入领域知识和验证通过的参考代码。所有组件通过函数聚合为最终的。

第二张图展示了模块依赖。三层拓扑结构用不同颜色清晰区分：绿色的只有一个叶子节点；蓝色的包含和两个互相独立的节点，它们都依赖，可以并行执行；黄色的是顶层汇聚节点，依赖前面所有子模块，拥有六个。

知识注入的核心在于预填充机制：从文件中检索验证通过的参考和代码，通过写入的工作区。这将的任务从从零生成正确的代码降级为基于验证通过的参考代码进行适配，显著降低了生成难度。实验结果也证实了这一策略的有效性：全部四个模块均在首次生成后直接通过验证，修复轮次为零。

4.4 批次调度机制 (Slide 16)

第十六页通过两张流程图详细展示了拓扑批调度的实现。

第一张是批次执行主循环的流程图。循环从收集当前状态（读取各模块的验证结果和晋升记录）开始，进入评估环节。如果所有模块都已成功，则终止并输出；如果有模块被阻塞且无法恢复，则终止并输出；否则继续选择下一个可执行的。选中后，首先检查其上游依赖是否已全部晋升：如果没有，则迟到下一次迭代；如果已经具备条件，且所有单模块都已晋升但集成测试尚未通过，则构建集成回归批次。准备就绪的进入

执行，完成后进行后处理 验证产出文件和执行 。如果 执行失败，则级联阻塞其所有下游依赖模块；如果成功，则检查预算后回到循环起点。

第二张是 评估的完整决策树。 函数首先调用 收集所有必需模块的状态，将每个模块分类为 (已晋升)、 (被阻塞)、 (级联阻塞) 或 (待处理)。然后从 中读取集成测试状态，最后进入六条分支的决策逻辑：成功完成、协议违规阻塞、必需模块阻塞、继续执行下一批次、就绪进入集成测试、无可批次阻塞。决策结果被写入 文件。

这里体现了一个重要的设计原则：文件系统门控。每个 的 通过 条件不是 说通过就通过，而是框架检查 目录下是否存在对应模块的验证通过文件。这是框架控制的具体体现 由文件系统事实而非 输出决定批次 进。

4.5 子功能模块实现 (Slide 17)

第十七页展示了 验证案例中四个子模块的具体实现细节。 实现了规范的 替换表，将 位输入映 到 位输出，是纯组合逻辑，通过检查点验证。 实现了 位密钥的 轮轮密钥扩展，通过 与 已知答案测试向量对比验证。 实现了单轮变换的四个步骤 、 、 和 ，通过 验证变换结果。顶层模块 实例化以上三个子模块，采用迭代微架构实现完整的 轮 加密， 个时钟周期完成一次加密并产生单周期 脉冲，拥有六个 覆盖 的完整生命周期。

4.6 MCP Tool 仿真 (Slide 18)

第十八页说明了 如何与 仿真器交互。核心组件是 一个异步 适配器，负责将框架的验证请求转换为 的编译和仿真操作。 通过调用 自定义工具发起请求（这是唯一合法入口）， 根据任务类型分发到对应的执行器，执行器通过 发起 调用。 负责预处理工作 过滤非文件、自动生成 包含路径标志、添加 编译标志，然后通过 管道与 通信，调用 和 两个工具。仿真完成后，从 中解析 行，生成结构化的验证结果。

层还实施了工具白名单过滤：只允许 和 两个工具，屏蔽了 和 等存在风险的工具入口，防止 绕过框架的结构化流程。

4.7 四层执行器与修复循环 (Slide 19)

第十九页是项目实施部分的收尾，通过两张流程图展示了四层分级验证执行器和结构化修复循环。

第一张图展示了修复循环的完整流程。当验证失败后，系统首先选择修复目标（文件或寄存器），然后生成修复请求（包含基线哈希和约束条件），派发给验证器。验证器读取请求后编辑目标文件，编辑完成后框架进行验证，检查文件哈希是否发生变化以及必需的资源是否存在。如果验证通过，记录修复并重新运行验证；如果重新验证也通过，则晋升工作区；如果任何一步失败，检查修复预算是否耗尽，有余量则继续循环，耗尽则标记为待修复。图中用黄色背景突出标识，表示这是用户操作的区域；用蓝色边框标识，表示这是框架控制的验证节点。哈希验证的设计确保了修复编辑确实修改了目标文件，防止出现声称已修复但实际未做任何更改。

第二张图展示了验证器工具的完整分发流程。调用验证器后，系统根据模式分发到四条不同的路径：模式初始化工作区并预填充寄存器，然后编译仿真；模式直接运行验证；模式记录并验证修复编辑；模式执行集成回归，运行验证器、验证器和验证器三个工具。所有涉及编译和仿真的路径最终都通过验证器执行，验证通过则晋升工作区，失败则标记待修复。

四层执行器形成了逐级递进的验证体系：编译门控在秒级时间内快速拦截语法错误和类型不匹配；在编译成功后执行仿真并解析寄存器，验证功能正确性；使用随机向量和边界条件进行鲁棒性测试；在所有模块晋升后运行三种验证器验证全系统的正确性和可靠性。

五、第四章：总结与展望（Slides 20-22）

5.1 LED 流水灯案例（Slide 21）

为了验证框架的通用性，汇报首先展示了一个简单案例——流水灯。这是一个单模块设计，仅有一个节点，没有模块间依赖。通过编写验证器，框架无需修改核心代码即可适配这个新设计，验证了蓝图驱动机制的通用性、状态机的基本流转以及验证链路的端到端工作能力。这个最小可验证设计证明了框架的核心流程不依赖于验证器的特定实现。

5.2 AES-128 全系统案例（Slide 22）

加密全系统是本课题的核心验证案例。实验结果全面而有力：在单模块验证方面，全部四个模块均在首次生成后直接通过编译与验证，修复轮次为零，首次通过率100%。在集成回归测试方面，使用（标准向量）、（连续多帧加密）和（加密过程中异步复位）三个测试用例全部通过，顶层模块六个寄存器无一遗漏。在框架控制有效性方面，10个执行批次中有1个修复批次因

（首次已通过）被自动跳过，所有 进决策均由状态机和 判定驱动，始终无法绕过验证门控。

六、核心创新点与整体评价

6.1 三大创新点

本课题针对现有 方案的五个不足（ 控制不可靠、线性 无法处理依赖、缺乏结构化修复、验证层级单一、 约束工程不足），提出了三个核心创新点。

第一个创新是框架控制的编排状态机。通过将控制权从 转移到十状态的编排状态机，配合 实现精确的行为边界控制，每个状态映 到确定的 配置和子委派策略。全局决策阶段使用 理配置以获得更强的 理能力，局部执行阶段使用配置以获得更 的响应速度。

第二个创新是 到 到 再到 的四层分级验证体系。从编译门控到功能仿真到鲁棒性测试再到集成回归，层层门控递进， 协议实现了机器自动判定，消除了对人工审查的依赖。

第三个创新是 六层 约束引擎。 层注入领域知识、 层通过可组合架构控制行为边界、 层将生成任务降级为适配任务、 层通过 哈希验证修复编辑、 层通过工具白名单过滤危险入口、 层提供带 预览的 命令。六层协同覆盖了幻觉端口、语法错误、 、过度修改和格式错误五种典型失败模式，相比仅依赖单一 的传统方案实现了质的飞跃。

6.2 整体评价

从演示文稿的质量来看，内容完整性和逻辑连贯性方面表现优秀，四章结构从背景到技术到实施到结果形成了严密的叙事闭环。技术深度方面，对 架构、 协议集成和状态机设计等核心技术均有深入到数学公式和变量维度层面的分析。创新展示方面，三大创新点均有明确的图示支撑和实验数据验证。

建议可进一步优化的方面包括： 中存在 总结与总结 的文字笔误应修正为 总结与展望； 的 数据流目前主要为文字描述，建议补充完整的数据流图； 的实验结果建议补充全流程执行时间和 消耗量等量化指标；此外， 中提到的三张 补充架构图（ 数据流、 六层架构、核心创新点循环关系）建议实际插入到演示文稿中，以进一步增强技术表达的完整性。

6.3 关键数据汇总

整个项目涉及四个功能模块、三层拓扑结构、十个状态机状态、九个验证、五种角色、十六个技能文件、四层验证执行器、三种集成回归和六层约束引擎。在全系统验证中实现了的首次通过率和零修复轮次，三个集成回归全部通过。模型总参数量为，其中模块占比。这些数据共同证明了本课题所提出的多自动化设计框架在正确性、可靠性和通用性方面的有效性。